Inclusify — Technical Guide
Installation & Maintenance
Group G07

Shahaf Wieder
Barak Sharon
Rasha Daher
Lama Zarka
Adan Daxa

Version 1.0
August 2026

Table of Contents

1. Introduction and Scope

Inclusify is an NLP-based web platform developed for the Achva LGBT organization. It analyzes academic texts in Hebrew and English and identifies language that may be LGBTQphobic, outdated, biased, potentially offensive, or pathologizing. The platform provides severity-graded findings, explanations, inclusive alternatives, and downloadable reports.

This Technical Guide explains how to install, configure, operate, monitor, and recover Inclusify after handover. It is written for a technically competent maintainer who is comfortable with a terminal, Docker, GitHub, PostgreSQL, and cloud dashboards, but who has no prior knowledge of the codebase.

Scope. The guide covers local installation with Docker Compose; production provisioning on Railway, Modal, and Cloudflare R2; environment configuration; routine maintenance; health and cost monitoring; troubleshooting; smoke testing; and backup and recovery. End-user workflows belong in the User Guide, while code modification and extension belong in the Development Guide.

Security rule - no secrets
Never place real credentials, API keys, OAuth secrets, database passwords, account passwords, tokens, or recovery codes in this document or its screenshots. Use placeholders in examples and deliver real values separately through the project's Bitwarden vault (see Section 3).

Related document	Purpose
User Guide	Explains how regular users and administrators use Inclusify.
Development Guide	Explains the repository structure, architecture, testing, and safe code changes.
Technical Guide	Explains installation, production operation, maintenance, verification, and recovery.

2. System Architecture

The current production stack contains three managed layers: Railway hosts the frontend, backend, PostgreSQL, and Redis; Modal serves the fine-tuned model on a serverless T4 GPU; and Cloudflare R2 stores documents, extracted text, and reports through an S3-compatible API. Google OAuth and Resend remain external services.

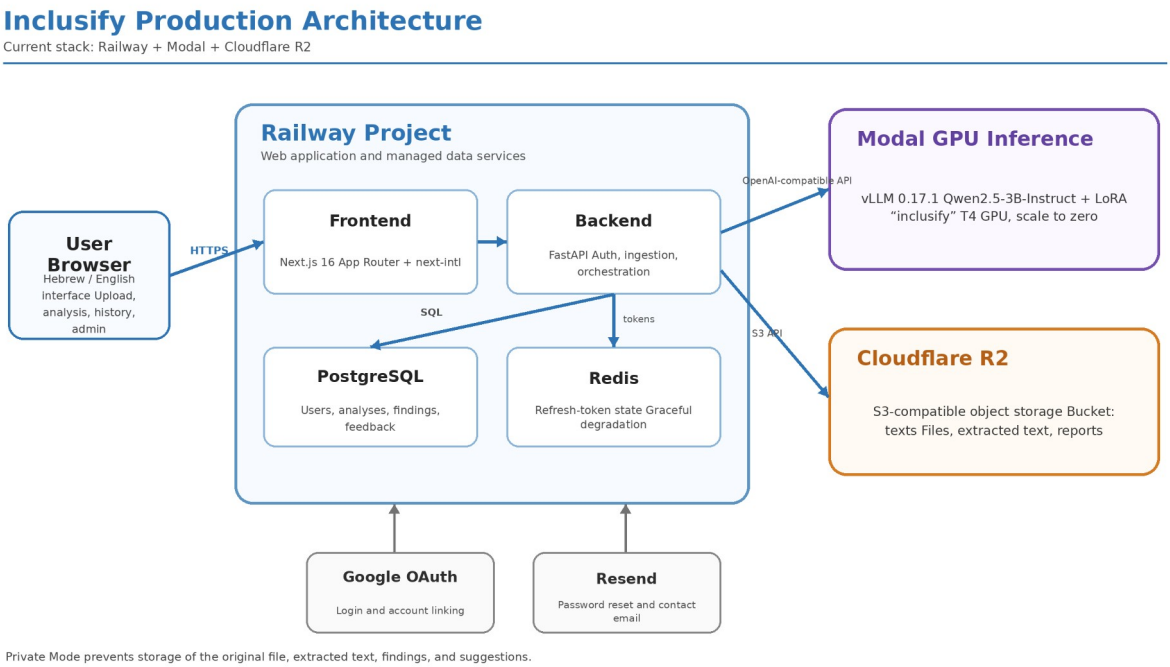

Figure 1. Current production architecture for Inclusify.

Production endpoints. Frontend: <FRONTEND_URL>. Backend: <BACKEND_URL>. Model endpoint: <MODAL_ENDPOINT>.

Component	Responsibility	Production location
Frontend	Next.js 16 application with bilingual routing, authentication screens, document analysis UI, history, profile, glossary, and admin dashboard.	Railway frontend service
Backend	Asynchronous FastAPI service for authentication, upload validation, Docling extraction, model orchestration, persistence, email, and health endpoints.	Railway backend service
PostgreSQL	Stores users, documents, analysis runs, findings, suggestions, feedback, and model metrics.	Railway managed PostgreSQL
Redis	Stores refresh-token state. The application continues with reduced authentication capability if Redis is temporarily unavailable.	Railway managed Redis
Modal vLLM	Serves Qwen/Qwen2.5-3B-Instruct with LoRA adapter qwen_r8_d0.2_achva_v2 under model name inclusify.	Modal app inclusify-vllm on T4 GPU
Cloudflare R2	Stores uploaded files, extracted text, and report objects when Private Mode is off.	R2 bucket texts
Google OAuth	Provides Google login and account linking.	Google Cloud OAuth client
Resend	Sends password-reset and contact-form email.	Resend account

2.1 Analysis Request Flow

A user uploads a supported document through the frontend. The backend validates the file type and 50 MB limit, parses the document through Docling, extracts metadata and chunks, detects the language, and preserves character offsets. The chunks are then sent to the Modal vLLM endpoint. The backend parses the returned JSON, maps severity and confidence, removes duplicate or overlapping findings, maps the surviving findings back to the source text, and calculates the Inclusivity Score.

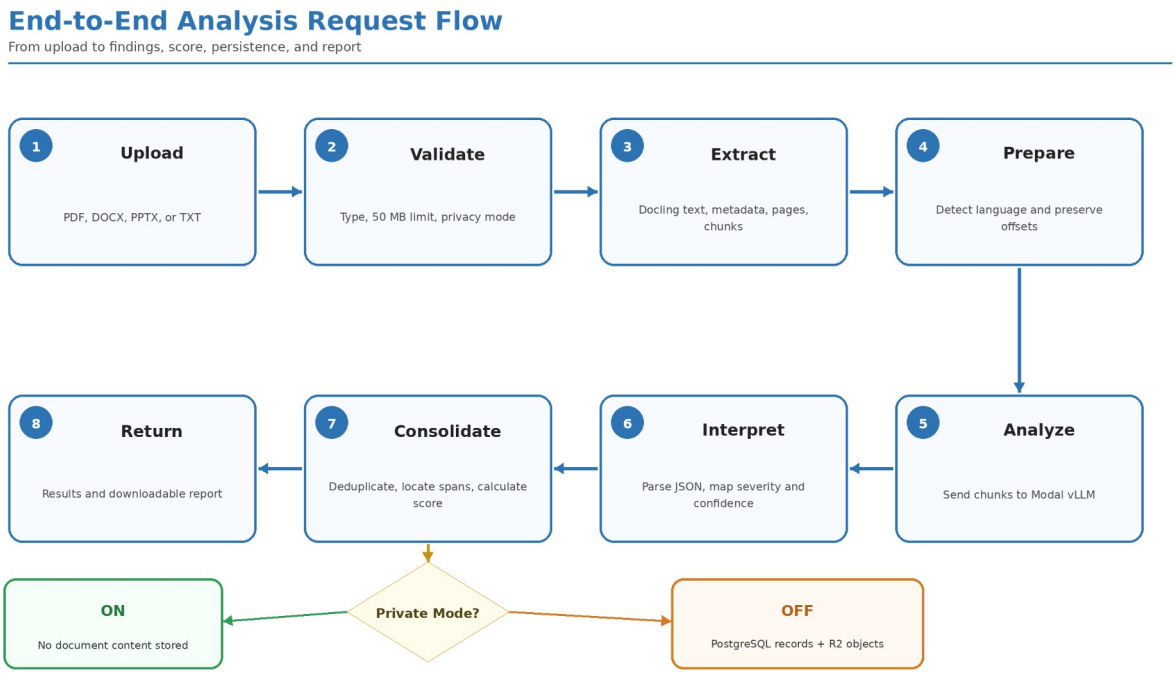

Figure 2. End-to-end document analysis flow, including the Private Mode persistence decision.

When Private Mode is enabled, the original file, extracted text, findings, and suggestions are not stored. Privacy-safe model-performance metrics may still be recorded. When Private Mode is disabled, the original file and extracted text are stored in R2, while the document, analysis run, findings, and suggestions are persisted in PostgreSQL.

3. Prerequisites and Accounts

The maintainer needs access to six service accounts and a small set of local tools. All cloud accounts should be Achva-owned after handover. Shared organizational addresses are preferable to personal addresses so ownership survives staff changes.

Account	Purpose	Minimum requirement
GitHub	Hosts the public repository and triggers Railway deployments.	Achva organization ownership; maintainer write/admin access
Railway	Hosts frontend, backend, PostgreSQL, and Redis.	Hobby plan, approximately $5/month, with a backend replica sized for its current memory requirement (around 4 GB)
Modal	Hosts the serverless T4 vLLM application.	Workspace ownership, billing method, Modal CLI token
Cloudflare	Hosts the R2 bucket and S3-compatible credentials.	Account administrator access and R2 enabled
Google Cloud	Hosts OAuth credentials for Google login.	Access to the Inclusify OAuth client
Resend	Sends password reset and contact-form email.	API key and a verified sending domain for unrestricted delivery

Local tool	Used for	Verification
git	Clone the repository and inspect changes.	git --version
Docker Desktop / Docker Engine	Run the complete local stack.	docker --version; docker compose version
Modal CLI	Deploy or inspect the model service.	modal --version
Railway CLI	Inspect services, logs, and database access.	railway --version
psql / pg_dump / pg_restore	Initialize, inspect, back up, and restore PostgreSQL.	psql --version
curl	Check health endpoints and API responses.	curl --version
rclone (optional)	Copy or back up R2 objects.	rclone version

Credential location. All production credentials and account-recovery details are stored in the project's Bitwarden vault, owned by an Achva organization account so that access survives staff changes. The vault holds the Inclusify support mailbox credentials, every production environment variable value, and the recovery details for each cloud account. No production secret is committed to Git or written in this guide.

4. Local Installation - Docker Compose

Use this procedure when a grader or maintainer needs to run Inclusify from a clean checkout. Docker Compose starts the frontend, backend, PostgreSQL, Redis, MinIO, and the one-shot MinIO initialization service. MinIO is the local substitute for Cloudflare R2.

4.1 Start the Development Stack

1. Install Git and Docker Desktop or Docker Engine with the Compose plugin.

2. Clone the official repository and enter the repository root.

3. Copy .env.example to .env. The template is fully commented and is configured for Docker service hostnames.

4. Review the local defaults. Change JWT_SECRET when testing authentication. Configure VLLM_URL and VLLM_API_KEY only when a reachable model endpoint is available.

5. Start the development profile and wait for the health checks.

6. Open the frontend and health endpoints listed in Section 4.2.

```
git clone https://github.com/achvalgbt/Inclusify.git
cd Inclusify
cp .env.example .env

docker compose --profile dev up
```

You should see:
PostgreSQL becomes healthy, Redis responds to ping, MinIO creates the texts bucket, the backend starts on port 8000, and the frontend becomes available on port 3100. The first startup can be slower because images and dependencies may need to be downloaded.

4.2 Local Services and Ports

Service	Host address	You should see
Frontend	http://localhost:3100	The Inclusify landing page loads.
Backend root	http://localhost:8000/	Returns “Inclusify API is running” and status OK.
Backend health	http://localhost:8000/health	Returns healthy when PostgreSQL is reachable.
Model health	http://localhost:8000/api/v1/health/model	Shows model status and circuit-breaker state.
PostgreSQL	localhost:5433	Host tools connect to port 5433; containers use postgres:5432.
Redis	localhost:6380	Host Redis clients use 6380; containers use redis:6379.
MinIO API	http://localhost:9000	S3-compatible API is reachable.
MinIO console	http://localhost:9001	Local object-storage console opens.

4.3 Database Initialization and Verification

On the first start of a new PostgreSQL volume, Docker mounts db/schema.sql and db/seed.sql into the database initialization directory. The files are applied automatically and are not re-applied when the volume already exists.

```
# Check containers
docker compose ps

# Backend deep health check
curl -s http://localhost:8000/health

# Model-health contract
curl -s http://localhost:8000/api/v1/health/model
```

Working without a model endpoint
The frontend, backend, database, Redis, and MinIO can run without vLLM. Upload and most interface flows remain testable, but actual analysis requires a reachable VLLM_URL. In Modal scale-to-zero mode, the first analysis after idle can take approximately 2-3 minutes while the GPU starts.

4.4 Production-Like Local Build

Use the production profile to verify the optimized runtime images. This catches Docker build failures, Docling warm-up failures, and frontend build-time configuration mistakes before cloud deployment.

```
BUILD_TARGET=runtime docker compose --profile prod up
```

Do not confuse host and container ports
The authoritative host ports are 3100, 8000, 5433, 6380, 9000, and 9001. Inside the Docker network, services continue to use their normal container ports and service names.

5. Production Installation - Cloud

Provision production in the order shown below. Each stage is independently verifiable. Do not move to the next stage until the current one has a clear success check.

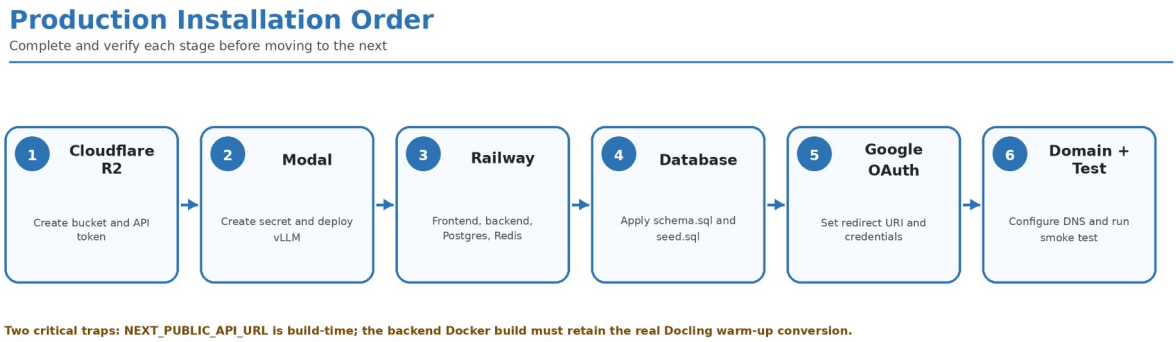

Figure 3. Required production installation order.

5.1 Cloudflare R2

1. Create an R2 bucket named texts.

2. Create an R2 API token scoped to the required bucket operations.

3. Record the endpoint in the form https://<ACCOUNT_ID>.r2.cloudflarestorage.com.

4. Store S3_ENDPOINT_URL, S3_ACCESS_KEY_ID, S3_SECRET_ACCESS_KEY, S3_BUCKET, and S3_REGION in the credential vault.

5. Use S3_BUCKET=texts and S3_REGION=auto.

You should see:
The bucket exists, the API token is stored securely, and a test S3 client can list or write objects without exposing the keys in screenshots.

5.2 Modal vLLM

1. Install the Modal CLI and authenticate the maintainer account.

2. Create the Modal secret inclusify-vllm-key. Its VLLM_API_KEY value must exactly match the backend VLLM_API_KEY.

3. Deploy infra/modal/vllm_app.py.

4. Record the full endpoint URL and verify that the app is listed as deployed.

```
pip install modal
modal token new
modal secret create inclusify-vllm-key VLLM_API_KEY=<VLLM_API_KEY>
modal deploy infra/modal/vllm_app.py
modal app list
```

Production Modal endpoint: <MODAL_ENDPOINT>.

Cost guardrail
Do not set min_containers to 1 or higher. The Modal app must be allowed to scale to zero when idle. Keep MODEL_SCALE_TO_ZERO=true on the backend so the health endpoint does not wake the GPU.

5.3 Railway Services

1. Create a Railway project in the Achva-owned workspace.
2. Add managed PostgreSQL 16 and Redis services.
3. Add the backend service from the public GitHub repository. Build it with infra/docker/backend.Dockerfile using the runtime target.
4. Add the frontend service from the same repository. Build it with infra/docker/frontend.Dockerfile.
5. Configure backend and frontend variables according to Section 6.
6. Confirm that both application services deploy from the main branch.

```
railway init
railway add --database postgres
railway add --database redis
```

Critical trap 1 - frontend API URL
NEXT_PUBLIC_API_URL is embedded into the Next.js bundle at build time. When the backend URL changes, update the frontend build variable and rebuild the frontend. Restarting the existing build is not sufficient.

Critical trap 2 - Docling warm-up
The backend runtime Dockerfile performs a real Docling conversion during the image build, then enables HF_HUB_OFFLINE=1 and TRANSFORMERS_OFFLINE=1. Do not remove or replace the real conversion with object construction; otherwise the first production upload may fail because the required models are not baked into the image.

5.4 Initialize the Production Database

Apply the canonical schema and seed files once against a fresh PostgreSQL service. The project does not use a migration framework. For an existing database, apply only reviewed, targeted SQL changes rather than re-running the full schema blindly.

```
railway run --service postgres psql < db/schema.sql
railway run --service postgres psql < db/seed.sql
```

You should see:
The commands finish without SQL errors, the expected tables exist, and the backend /health endpoint reports the database component as healthy.

5.5 Google OAuth and Public Domains

1. Open the Inclusify OAuth client in Google Cloud.
2. Add https://<backend-domain>/auth/google/callback as an authorized redirect URI.
3. Set GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET on the Railway backend service.
4. Add the frontend custom domain to Railway and copy both the CNAME and TXT records into the Achva Wix DNS panel. Both records are required.
5. Set FRONTEND_URL to the public frontend URL and add that URL to ALLOWED_ORIGINS.
6. Keep the backend on its Railway public domain unless the team deliberately creates a separate backend custom domain.
7. Do not embed the application in an iframe; use a normal link from the Achva website.

Setting	Value to enter after confirmation
Frontend public URL	<FRONTEND_URL>
Backend public URL	<BACKEND_URL>
Google authorized redirect URI	<BACKEND_URL>/auth/google/callback
FRONTEND_URL	<FRONTEND_URL>
ALLOWED_ORIGINS	Include <FRONTEND_URL>; retain old origin during cutover if needed
NEXT_PUBLIC_API_URL	<BACKEND_URL> - frontend build variable

5.6 Production Readiness Checkpoints

Checkpoint	Evidence of success
Cloudflare R2	Bucket texts exists; token is stored in the vault; no keys appear in the document.
Modal	inclusify-vllm is deployed; endpoint recorded; scale-to-zero preserved.
Railway backend	Deployment succeeds; /health returns HTTP 200 with database healthy.
Railway frontend	The app loads and calls the intended backend.
Database	Fresh schema and seed completed without errors.
OAuth	Google login completes and returns to the correct frontend.
End-to-end	Upload, analysis, report download, and contact email all pass.

6. Configuration Reference

The following tables combine the authoritative settings class with variables read directly by the application and Docker configuration. Development defaults are shown where they exist. Production secrets must be stored in the credential vault and entered only in the appropriate dashboard.

6.1 Database and Redis

Variable	Default / example	Description
DATABASE_URL	unset	Optional complete database URL. If absent, the backend constructs one from the PG* variables.
PGHOST	postgres in Compose	PostgreSQL host. Railway normally injects a service reference.
PGPORT	5432 in container	PostgreSQL port. The local host mapping is 5433.
PGDATABASE	inclusify	Database name.
PGUSER	postgres	Database user.
PGPASSWORD	devpassword locally	Database password. Use a managed production secret.
PGSSL	empty locally; require in managed production	SSL mode for the PostgreSQL connection.
REDIS_URL	redis://localhost:6379	Redis URL. Compose backend uses redis://redis:6379.

6.2 Authentication, OAuth, Frontend, and CORS

Variable	Default / example	Description
JWT_SECRET	dev-secret-change-in-production	JWT signing secret; replace with a long random production value.
ACCESS_TOKEN_EXPIRE_MINUTES	43200	Access-token lifetime in minutes.
REFRESH_TOKEN_EXPIRE_DAYS	30	Refresh-token lifetime in days.
GOOGLE_CLIENT_ID	empty	Google OAuth client ID.
GOOGLE_CLIENT_SECRET	empty	Google OAuth client secret.
FRONTEND_URL	http://localhost:3000	Backend view of the frontend for redirects and generated links. Production: <FRONTEND_URL>.
ALLOWED_ORIGINS	local frontend origins	Comma-separated CORS allow-list. Production must include <FRONTEND_URL>.
NEXT_PUBLIC_API_URL	http://localhost:8000	Frontend build-time backend URL. Production: <BACKEND_URL>.

6.3 Model Service and Circuit Breaker

Variable	Default	Description
VLLM_URL	http://localhost:8001	Model endpoint. Production uses <MODAL_ENDPOINT>.
VLLM_API_KEY	empty locally	Bearer token. Must match Modal secret inclusify-vllm-key.
VLLM_MODEL_NAME	inclusify	LoRA module name sent in every model request.
VLLM_TIMEOUT	240.0 seconds	HTTP timeout wide enough for current Modal cold starts.
VLLM_READY_BUDGET	240.0 seconds	Maximum time /analyze may wait for the scale-to-zero GPU to become ready.
VLLM_MAX_CONCURRENT	16	Application concurrency ceiling; matches vLLM max-num-seqs.
VLLM_CIRCUIT_FAIL_MAX	5	Consecutive failures before the circuit opens.
VLLM_CIRCUIT_RESET_TIMEOUT	60 seconds	Time before the circuit breaker attempts recovery.
VLLM_LOAD_TEST_MODE	false	Synthetic responses for load testing only; never enable in normal production.
MODEL_SCALE_TO_ZERO	false by default; true on Modal	Prevents health polling from waking a serverless GPU.
MODEL_HEALTH_CACHE_TTL	300.0 seconds	Cache duration for live probes when the model is always on.

6.4 Storage and Email

Variable	Default / example	Description
S3_ENDPOINT_URL	http://minio:9000 locally	R2 endpoint in production: https://<ACCOUNT_ID>.r2.cloudflarestorage.com.
S3_ACCESS_KEY_ID	minioadmin locally	S3/R2 access key ID.
S3_SECRET_ACCESS_KEY	minioadmin locally	S3/R2 secret access key.
S3_BUCKET	texts	Object-storage bucket name.
S3_REGION	auto	Required value for Cloudflare R2; ignored by MinIO.
RESEND_API_KEY	empty	Resend API key. Empty means email sends are skipped.

Variable	Default / example	Description
RESEND_FROM	Inclusify <onboarding@resend.dev>	Sender for password-reset and contact email. Production must use a Resend-verified domain.
CONTACT_RECIPIENTS	project inbox / admin fallback	Comma-separated contact-form recipients. If empty, the application can fall back to site administrators.

6.5 Operational and Build Variables

Variable	Default / example	Description
LOG_LEVEL	INFO	Controls application and Uvicorn verbosity. INFO/DEBUG go to stdout; WARNING+ go to stderr.
BLOCK_OCR_DOCUMENTS	false	When true, rejects scanned PDFs and disables OCR to protect memory-constrained hosts.
BUILD_TARGET	development locally	Selects development or runtime Docker stages.
BUILD_TIME	empty locally	Build timestamp exposed by the health endpoint when CI populates it.
GIT_COMMIT	empty locally	Commit identifier exposed by the health endpoint when CI populates it.
WATCHPACK_POLLING	true in Compose frontend	Improves file watching for Docker Desktop and WSL development.
CHOKIDAR_USEPOLLING	true in Compose frontend	Additional frontend hot-reload polling setting.

Not environment variables

The /analyze rate limit of 20 requests per hour per caller and the 200,000-character request cap are defined in code, in backend/app/modules/analysis/router.py, rather than through configuration. Uvicorn runs with --proxy-headers and --forwarded-allow-ips=* so that per-IP limits observe the real client address rather than the platform edge proxy.

Configuration validation

After any production variable change, redeploy or restart the relevant service, then run the health checks and the affected smoke-test step. Changes to NEXT_PUBLIC_API_URL require a frontend rebuild.

7. Routine Maintenance

7.1 Deploy Application Updates

1. Create and review a pull request.

2. Confirm GitHub Actions linting and tests pass.

3. Merge or push the approved change to main.

4. Watch the Railway frontend and backend deployments.

5. Run the focused health checks and smoke-test steps affected by the change.

6. Use the Railway Deployments menu to redeploy a previous build if a rollback is required.

Temporary - automatic deployment is pending on Achva's side
Automatic deployment from main is paused at the time of handover. The repository moved to the achvalgbt organization, and a Railway GitHub App installation does not follow a repository to a new owner. Continuous deployment worked throughout the project under the team's own management, and it resumes as soon as an achvalgbt organization owner installs the Railway GitHub App and reconnects the backend and frontend services to branch main; the project team remains available to support Achva in completing this step. Until then, deploy from a clean checkout of main with railway up -s backend -e production -d and the same command for frontend. Run it from the repository root, because railway up uploads the git root rather than the current directory.

```
gh run list --workflow CI --branch main --limit 5
gh run view <RUN_ID> --log-failed
```

7.2 Deploy Model or Serving Changes

Model-serving changes are not deployed automatically by Railway. Update and validate infra/modal/vllm_app.py or the selected adapter, then deploy manually.

```
modal deploy infra/modal/vllm_app.py
modal app logs inclusify-vllm
```

7.3 Read Logs

```
railway logs --service backend
railway logs --service frontend
modal app logs inclusify-vllm
```

The backend intentionally sends INFO and DEBUG records to stdout and WARNING or higher to stderr, because Railway derives severity from the output stream. Do not replace this with basic logging without understanding the monitoring impact.

7.4 Database Operations and Admins

There is no migration framework. Keep db/schema.sql canonical for new installations and apply reviewed schema changes to the live database as targeted SQL. Everyday role management should use the Admin dashboard; SQL remains an operational fallback.

```
railway connect postgres

SELECT email, role FROM users;
UPDATE users SET role = 'site_admin' WHERE email = 'user@example.com';
```

Change discipline
For every live schema change: update db/schema.sql, write the equivalent targeted production SQL, apply it

deliberately, document it in the pull request, and verify both a fresh installation and the live database.

8. Monitoring, Health, and Cost Guardrails

8.1 Health Endpoints

Check	Command / endpoint	Healthy interpretation
Backend and database	GET <BACKEND_URL>/health	HTTP 200; overall status healthy; database healthy; pool statistics present.
Model health	GET <BACKEND_URL>/api/v1/health/model	HTTP 200; status available; circuit breaker closed. In production, state may be scale_to_zero.
Railway services	railway status	Project linked and required services present.
Modal deployment	modal app list	inclusify-vllm appears deployed.
Model logs	modal app logs inclusify-vllm	Startup or inference logs appear without repeated failures.

8.2 Circuit Breaker and Cold Start

The vLLM circuit breaker opens after the configured number of consecutive failures and automatically attempts recovery after the reset timeout. With MODEL_SCALE_TO_ZERO=true, the model-health endpoint does not probe Modal; it infers availability from the circuit-breaker state so routine health polling cannot wake or keep the GPU warm.

The current Modal T4 cold start has been measured at approximately 162-183 seconds after scale-to-zero. VLLM_TIMEOUT and VLLM_READY_BUDGET therefore default to 240 seconds. The first analysis after idle may be slow without indicating an outage.

8.3 Cost Guardrails

- Never configure min_containers >= 1 on Modal unless the organization intentionally accepts always-on GPU cost.
- Keep MODEL_SCALE_TO_ZERO=true on the Railway backend when production inference runs on Modal.
- Do not poll the Modal endpoint directly from the frontend or an external uptime monitor.
- Keep the Modal scaledown window at five minutes unless a documented cost/latency decision changes it.
- Set a Modal spend alert and a Railway usage alert on the first day of handover.
- Review Railway replica memory before lowering resources; Docling parsing is memory intensive.

Budget warning
A continuously warm T4 GPU can cost approximately $430 per month even with low traffic. Scale-to-zero and non-probing health checks are mandatory cost controls, not optional tuning. At the expected usage level, steady-state cost should stay near the Railway Hobby plan (~$5/month) plus a single-digit-dollar Modal GPU charge; actual cost still depends on traffic and must be monitored with the alerts above.

9. Troubleshooting

Start with the symptom, check the listed evidence, and apply the least disruptive fix. Re-run the relevant health check after every change. Never “fix” an incident by exposing secrets, disabling security controls, or switching to unapproved infrastructure.

Symptom	Likely cause	Corrective action
Backend /health returns 503	PostgreSQL is unavailable, variables are wrong, or the pool cannot connect.	Check Railway Postgres status and PG* references; inspect backend logs; restart only after verifying variables.
Frontend cannot reach the backend	NEXT_PUBLIC_API_URL points to the wrong URL or CORS excludes the frontend origin.	Set the correct build variable, rebuild the frontend, and add the frontend URL to ALLOWED_ORIGINS.
Google login returns a redirect error	OAuth redirect URI does not match the active backend domain.	Add <BACKEND_URL>/auth/google/callback to the Google OAuth client and retest.
Model unavailable banner appears	Circuit breaker is open after real failures, or Modal deployment/configuration is invalid.	Check /api/v1/health/model, modal app list, Modal logs, VLLM_URL, and VLLM_API_KEY matching.
First analysis after idle is slow	Normal Modal scale-to-zero cold start.	Allow up to the 240-second ready budget. Do not wake the GPU with health probes.
Analysis still fails after several minutes	Incorrect endpoint/key, failed Modal startup, or model image problem.	Inspect Modal logs; verify <MODAL_ENDPOINT>; confirm the backend and Modal use the same API key; redeploy if needed.
Circuit breaker stays open	Repeated model failures continue or reset has not elapsed.	Fix the underlying Modal/credential issue, wait for reset timeout, then run one controlled analysis.
Users report "Too many analyses"	The caller exceeded 20 analyses per hour (per signed-in user, or per client IP for guests).	Expected protection for a paid GPU. The window clears one hour after the caller's oldest counted request. Adjust _RATE_LIMIT and _RATE_WINDOW in backend/app/modules/analysis/router.py and redeploy. The counter is per backend replica and resets on restart.

Symptom	Likely cause	Corrective action
Upload fails only in production	The Docling warm-up conversion was removed or did not complete before offline mode.	Restore the real build-time conversion in backend.Dockerfile and rebuild the backend image.
Scanned PDF rejected	BLOCK_OCR_DOCUMENTS=true or the host cannot safely run OCR.	Use a text-layer PDF, disable the guard only on a host with adequate memory, then monitor memory carefully.
Uploads queue for a long time	Parsing is deliberately serialized at one concurrent parse to bound memory.	Wait, reduce document size, or scale backend replicas deliberately; do not raise concurrency without memory testing.
Container is killed during OCR	Docling/Tesseract exceeded available memory.	Use the 4 GB backend profile, keep CPU-thread limits, or enable BLOCK_OCR_DOCUMENTS on smaller hosts.
Contact-form or reset email does not arrive	Resend key/sender is invalid, the domain is unverified, or free-tier delivery is restricted.	Check Resend send logs; use a verified domain and correct RESEND_FROM; do not switch to SMTP because Railway blocks it.
Railway displays normal logs as errors	The level-split logging configuration was removed or bypassed.	Restore the project logging configuration so INFO/DEBUG use stdout and WARNING+ use stderr.
Custom domain returns 404	Railway domain verification is incomplete.	Confirm both CNAME and TXT records exist in Wix DNS and wait for Railway to mark the domain Active.
Report or file retrieval fails	R2 endpoint, bucket, or credentials are wrong; object is missing.	Check S3_* variables, bucket texts, R2 permissions, and backend logs. Do not expose keys in screenshots.

Escalation record

For unresolved incidents, record the timestamp, affected URL, user-visible symptom, relevant HTTP status, Railway deployment ID, last successful change, sanitized log excerpt, and actions already attempted. Never include tokens or database URLs.

10. Verification — Smoke Test

Run this test after a fresh installation, domain change, infrastructure transfer, major deployment, model update, or recovery. Perform it as a normal user against the production frontend. Record evidence without capturing secrets or personal document content.

Step	Test	You should see
1	GET <BACKEND_URL>/health	HTTP 200; database component healthy.
2	GET <BACKEND_URL>/api/v1/health/model	HTTP 200; model available; circuit breaker closed or scale_to_zero state.
3	Open <FRONTEND_URL> and complete Google login	OAuth round-trip succeeds and returns to the frontend.
4	Upload a small PDF or DOCX with Private Mode off	Text extraction succeeds and an object is written to R2.
5	Run analysis	Findings or a valid zero-finding result return; Modal logs show inference.
6	Download the report	Report renders and downloads successfully.
7	Submit the contact form	Email arrives at a non-owner inbox using the verified Resend domain.
8	Push a harmless commit to main	Railway automatically builds and deploys the connected services.

10.1 Smoke-Test Record

Date / time	Tester	Frontend	Backend	Result	Notes / evidence
		<FRONTEND_URL>	<BACKEND_URL>		

Release rule
Do not declare installation, handover, or recovery complete until all eight steps pass. When a step fails, stop and troubleshoot that dependency before continuing.

11. Backup and Recovery

11.1 What Is Stateful

State	Location	Backup approach	Rebuildable from Git?
Users, analyses, findings, feedback, model metrics	Railway PostgreSQL	Railway backups plus periodic pg_dump verification	No
Uploaded files, extracted text, report objects	Cloudflare R2 bucket texts	R2 retention/object copy; optional rclone mirror	No
Frontend/backend code and Docker configuration	GitHub repository	Git history and repository ownership	Yes
Model-serving application and adapter	GitHub + Modal deployment	Redeploy infra/modal/vllm_app.py from the repository	Yes
Secrets and account recovery details	Credential vault and cloud dashboards	Vault backup and organizational ownership procedures	No - must be managed separately

11.2 PostgreSQL Backup and Restore

Use Railway's standard postgresql:// connection string with pg_dump and pg_restore. Store dumps in an encrypted, access-controlled location, verify the archive list, and restore only into a prepared target database.

```
pg_dump --format=custom --no-owner --no-acl --dbname="<RAILWAY_POSTGRES_URL>" --file="inclusify-
YYYY-MM-DD.dump"
pg_restore --list inclusify-YYYY-MM-DD.dump
pg_restore --clean --if-exists --no-owner --no-acl --dbname="<TARGET_POSTGRES_URL>" inclusify-
YYYY-MM-DD.dump
```

After restoration, run /health, confirm login and existing data, then complete the full smoke test.

11.3 R2 Object Backup and Recovery

Cloudflare R2 objects can be mirrored with rclone when the organization requires an independent copy. Configure credentials outside the repository and never paste the rclone configuration into this guide.

```
# Backup from R2 to an approved secure location
rclone sync r2:texts secure-backup:inclusify/texts --progress

# Restore after review
rclone sync secure-backup:inclusify/texts r2:texts --progress
```

11.4 Full Recovery Sequence

1. Confirm ownership and access to GitHub, Railway, Modal, Cloudflare, Google Cloud, Resend, and the credential vault.
2. Redeploy the backend and frontend from main using the production Dockerfiles.
3. Restore or reconnect PostgreSQL.
4. Restore or reconnect the R2 bucket and S3 credentials.
5. Redeploy the Modal vLLM application and confirm the endpoint.
6. Reapply Railway environment variables from the credential vault.
7. Confirm OAuth redirect URIs, frontend URL, CORS, and Resend sender configuration.
8. Run the full eight-step smoke test before reopening the service.